SKALA AI Mini-Project · 과제 1 (Base AI Agent 개선)

장애 대응 에이전트, 키워드에서 의미 검색으로

RAG 기반 runbook 판별 · 하드코딩 제거 · 근거 검증 · 모듈화 · 자동 평가

광주_1반_G030_최재은 | 2026.10.22

01 · 주제

운영 인시던트 대응 에이전트

운영자가 장애 증상을 채팅으로 물으면, 에이전트가 **모니터링 지표**와 **대응 매뉴얼(runbook)**을 직접 조사해 번호 절차로 조치를 답한다.



베이스 구성

- FastAPI + LangGraph StateGraph, SSE 스트리밍, Vue 채팅 UI
- MCP 서버 2개(ops/docs) — 모니터링 Tool, runbook Resource, 진단 Prompt
- LLM: OpenAI gpt-4o-mini(판단) / gpt-4o(답변 작성)

직접 돌려보며 찾은 약점

- runbook을 **키워드 첫 매칭 1개**로 선택, 모르는 표현은 전부 **db-down**
- 시스템명이 없으면 **무조건 auth-service** 진단
- 검증은 "30자 이상 + 1) 포함"만 확인
- graph.py 단일 파일, 중복 코드 3곳, 테스트 0개

02 · 기능 개선 및 추가 포인트

무엇을 바꾸고 무엇을 더했나

구분	항목	Before → After	위치
개선	**RAG 기반 incident 판별**	키워드 첫 매칭 1개 → runbook 임베딩 의미 검색, 저하된 시스템마다 판별	`agent/rag/` `classifier.py`
개선	**하드코딩 제거**	시스템명 없으면 auth-service → 시스템 카탈로그 검색, 미등록 시스템은 "모니터링 대상 아님"	`nodes/planning.py`
개선	**근거 검증**	형식만 검사 → 답변의 % 수치가 수집된 사실에 있는지 코드로 검사, 없으면 재작성	`nodes/writing.py`
개선	**모듈화**	graph.py 단일 파일 → 역할별 5개 모듈, 3곳 중복 코드 단일화	`agent/nodes/` `systems.py`
추가	**데이터 보강**	시스템 3종, 장애유형 2종(에러율 급증·인증서 만료), 시스템 카탈로그	`data/`
추가	**자동 평가**	라벨 24건 — 검색 정확도·지연·비용·retry + LLM-as-judge	`eval/`
추가	**단위 테스트**	28개, 가짜 MCP 주입으로 네트워크 없이 1초	`tests/`

runbook 판별: 키워드 규칙 → 의미 검색

Before — KeywordIncidentClassifier

```
text = 모든 시스템 알림을 한 문자열로 합침
for kw, type in RULES:      # latency, pool, cpu ...
    if kw in text: return [type]   # 첫 매칭 1개
return ["db-down"]          # 그 외 전부
```

After — RagIncidentClassifier

```
probes = [시스템별 알림 문장]  (없으면 사용자 질문)
hits = kb.search(probes, kind="runbook", k=1)  # 임베딩 1회 batch
→ 시스템마다 top-1 청크의 metadata.incident_type
→ 중복 제거, 최대 3개 runbook을 MCP Resource로 조회
```

지식베이스 (agent/rag)

- runbook 8종 → **요약 청크** + **절차 청크**, metadata에 incident_type
- text-embedding-3-small, 코사인 검색 (청크 55개라 외부 벡터DB 없이 구현)
- 인덱스 디스크 캐시: 키 = (임베더, 코퍼스 해시) → 재기동 시 비용 0, runbook 수정 시 자동 재색인
- 새 장애 유형 = **.md 파일 추가만** (코드 수정 없음)
- `INCIDENT_CLASSIFIER=keyword|rag` 토글 → 같은 코드로 A/B 측정

03 · 사용한 기술 요소

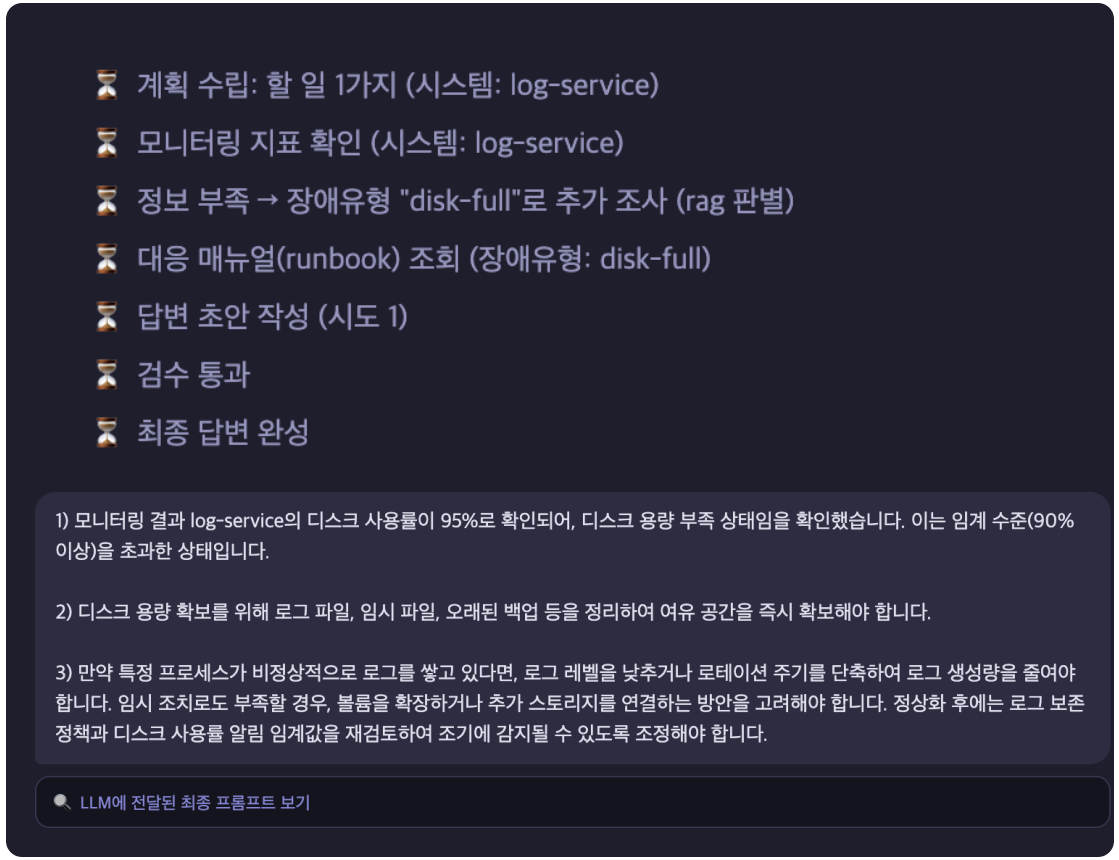
기술 요소별 적용 내용

기술	상태	이번 과제에서 한 일
RAG	신규	runbook·시스템 카탈로그 임베딩 검색. 검색 결과의 **메타데이터(incident_type)**만 쓰고 본문은 MCP로 조회
MCP	확장	Tool(모니터링)·Resource(runbook)·Prompt(진단 절차) 실호출. 보강한 시스템·runbook을 MCP 서버 데이터로 추가
의존성 주입	확장	`SupervisorDeps`에 판별기·시스템 해석기·검증 토글 추가 → 환경변수로 교체, 테스트에서 가짜 MCP 주입
모듈화	신규	graph.py는 배선만(52줄), 노드는 `agent/nodes/` 5개 파일, RAG는 `agent/rag/` 3계층
단위 테스트	신규	pytest 28개 — 노드·RAG·그래프 전체 경로, 무한 루프 방지·미등록 시스템 처리
Multi-Agent	기존 활용	Supervisor 그래프가 계획→조사→분석→작성→검수 역할에 task 분배, 판단(mini)·작성(4o) 모델 분리. RAG 판별기를 조사 단계에 연결
LangSmith	기존 유지	모든 노드·LLM 호출에 `@traceable`. API 키가 없어 실측은 자체 평가 도구(eval/)로 지연·비용·정확도를 기록

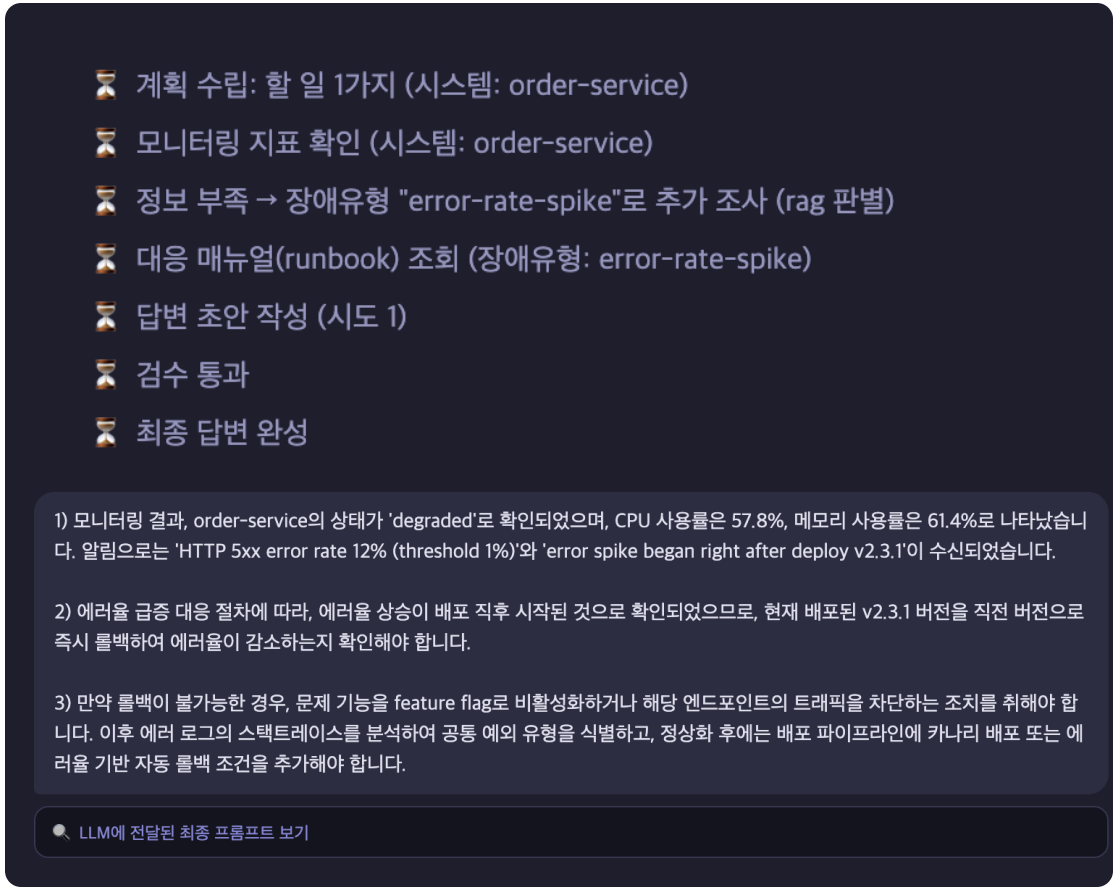
03 · 구현 기능 화면 ①

RAG 판별 — 시스템명 없는 질문, 신규 장애유형

"로그 저장 서버 용량이 꽉 찼대요" → 카탈로그 검색으로 **log-service**, RAG 판별로 **disk-full**



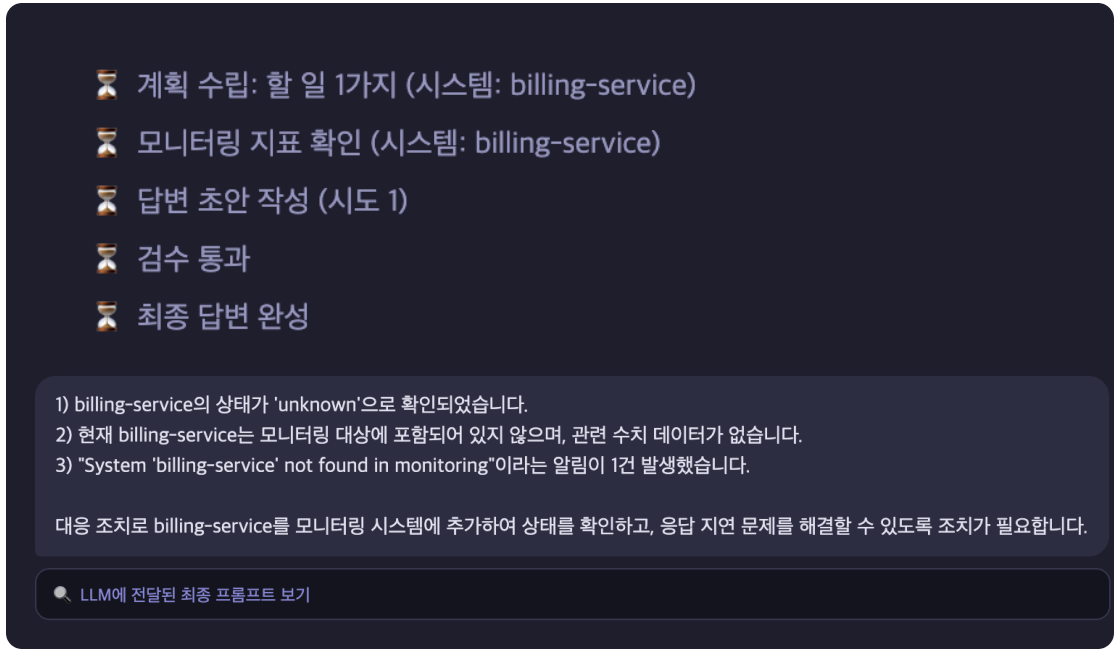
"order-service 배포하고 나서 에러가 엄청 나요" → 신규 runbook **error-rate-spike** (rag 판별)



03 · 구현 기능 화면 ②

미등록 시스템 처리 · 단위 테스트

"billing-service 응답이 느려요" → 다른 시스템을 대신 진단하지 않고 **모니터링 대상 아님**, runbook 생략



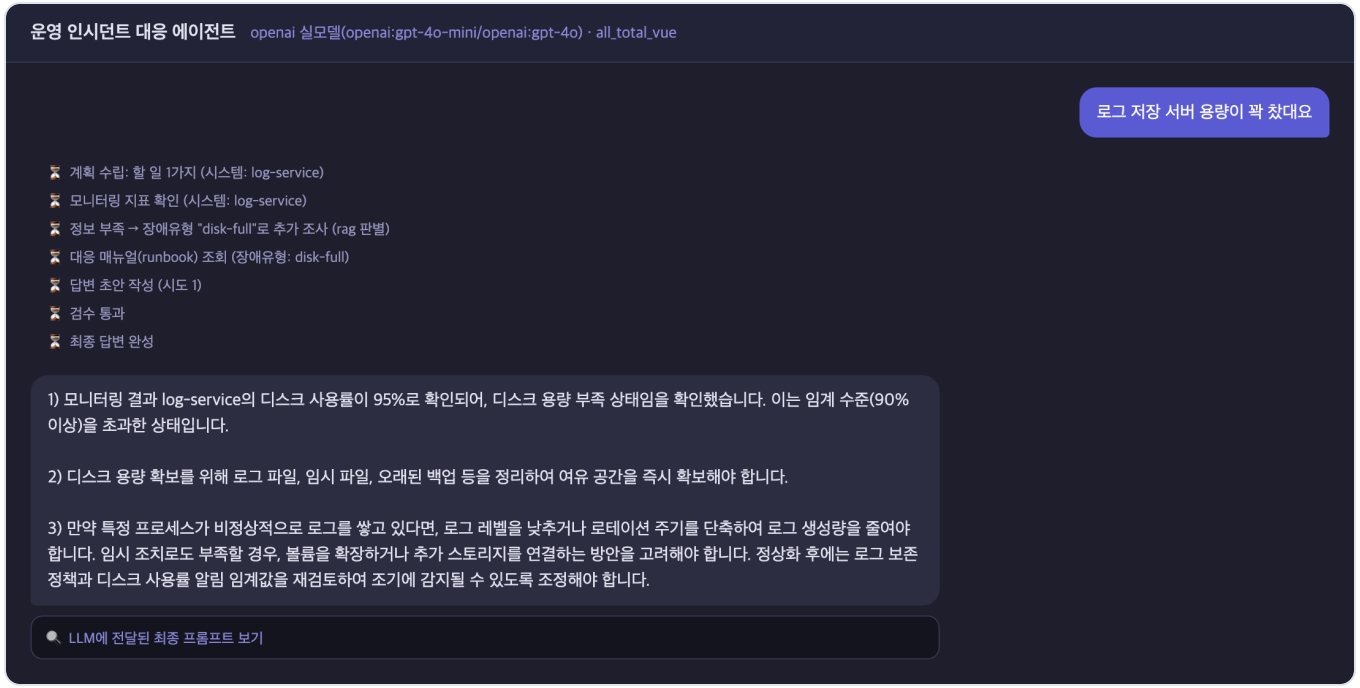

```
all_total_vue_student — zsh

$ .venv/bin/python3 -m pytest -v
tests/test_graph.py::test_chat_route_skips_pipeline PASSED
tests/test_graph.py::test_incident_pipeline_fetches_classified_runbooks PASSED
tests/test_nodes.py::test_gate_removes_contradicting_analysis_task PASSED
tests/test_nodes.py::test_replan_adds_gap_task_per_incident_type PASSED
tests/test_nodes.py::test_replan_stops_after_one_round PASSED
tests/test_nodes.py::test_plan_keeps_model_targets_when_system_mentioned PASSED
tests/test_nodes.py::test_plan_uses_resolver_when_no_system_mentioned PASSED
tests/test_nodes.py::test_plan_investigates_unknown_system_instead_of_default PASSED
tests/test_nodes.py::test_ungrounded_percentages PASSED
tests/test_nodes.py::test_validate_rejects_ungrounded_numbers PASSED
tests/test_nodes.py::test_validate_passes_after_max_attempts PASSED
tests/test_nodes.py::test_replan_without_verdicts_goes_straight_to_write PASSED
tests/test_rag.py::test_vector_store_search_and_filter PASSED
tests/test_rag.py::test_vector_store_cache_invalidates_on_key_change PASSED
tests/test_rag.py::test_runbook_chunks_carry_incident_type_metadata PASSED
tests/test_rag.py::test_knowledge_base_reuses_cached_index PASSED
tests/test_rag.py::test_keyword_classifier_keeps_legacy_behavior PASSED
tests/test_rag.py::test_rag_classifier_one_verdict_per_degraded_system PASSED
tests/test_rag.py::test_rag_classifier_falls_back_when_below_threshold PASSED
tests/test_rag.py::test_rag_classifier_uses_query_when_no_alerts PASSED
tests/test_rag.py::test_system_resolver PASSED
tests/test_rag.py::test_rag_classifier_skips_when_only_unknown_systems PASSED
tests/test_systems_keywords.py::test_known_systems_follow_monitoring_json PASSED
tests/test_systems_keywords.py::test_find_mentioned_systems_keeps_order PASSED
tests/test_systems_keywords.py::test_guess_falls_back_to_default PASSED
tests/test_systems_keywords.py::test_routing_keywords_catch_common_symptoms PASSED
tests/test_systems_keywords.py::test_routing_keywords_leave_small_talk PASSED
tests/test_systems_keywords.py::test_describe_metrics_hides_placeholder_for_unknown_system
PASSED
============================== 28 passed in 0.92s ==============================
```

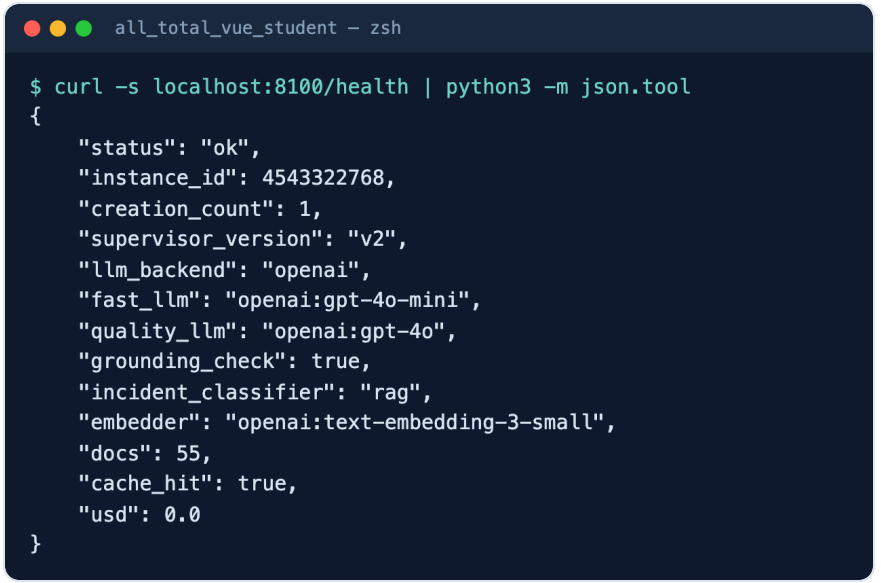
`pytest -v` — 28 passed, 0.9초 (네트워크·API 키 없이 실행)

03 · 최종 실행 화면

OpenAI 백엔드 + Vue 채팅 UI (localhost:8600)



채팅 UI — 진행 단계(계획·조사·RAG 판별·검수)가 SSE로 실시간 표시



서버 상태 — `incident_classifier: rag`, 청크 55개, 인덱스 캐시 사용

04 · BEFORE / AFTER — 같은 입력

“로그 저장 서버 용량이 꽉 찼대요”

Before 원본 auth-service를 조사하고 high-latency runbook으로 **엉뚱한 진단**

- ⏳ 계획 수립: 할 일 1가지 (시스템: auth-service)
- ⏳ 모니터링 지표 확인 (시스템: auth-service) (재시도 성공)
- ⏳ 정보 부족 → 장애유형 "high-latency"로 추가 조사
- ⏳ 대응 매뉴얼(runbook) 조회 (장애유형: high-latency)
- ⏳ 답변 초안 작성 (시도 1)
- ⏳ 검수 통과
- ⏳ 최종 답변 완성

1) 로그 저장 서버의 용량이 꽉 찬 문제를 해결하기 위해, 현재 auth-service의 상태를 점검한 결과 상태가 'degraded'로 확인되었습니다. CPU 사용률은 83.9%, 메모리 사용률은 63.9%이며, 두 건의 알림이 발생했습니다: 'p95 latency 2.4s (threshold 1.0s)'와 'connection pool 92% used'.

2) Runbook에 따라, p95 지연시간과 커넥션 풀 사용률을 확인한 결과, 커넥션 풀 사용률이 90%를 초과했습니다. 따라서, 커넥션 풀 크기를 늘리거나 느린 쿼리 및 외부 호출에 타임아웃을 설정하는 조치를 고려해야 합니다.

3) 응답 지연 문제를 해결하기 위해, 5분 내에 p95 지연시간이 임계치(1.0s) 아래로 내려오지 않으면 캐시 계층 또는 읽기 복제본으로 트래픽을 분산하는 방안을 실행해야 합니다. 이후, 정상화가 이루어지면 느린 쿼리 로그를 분석하여 근본 원인을 문서화하고 후속 최적화 티켓을 등록해야 합니다.

🔍 LLM에 전달된 최종 프롬프트 보기

After log-service · disk-full — 디스크 정리 절차로 답변

- ⏳ 계획 수립: 할 일 1가지 (시스템: log-service)
- ⏳ 모니터링 지표 확인 (시스템: log-service)
- ⏳ 정보 부족 → 장애유형 "disk-full"로 추가 조사 (rag 판별)
- ⏳ 대응 매뉴얼(runbook) 조회 (장애유형: disk-full)
- ⏳ 답변 초안 작성 (시도 1)
- ⏳ 검수 통과
- ⏳ 최종 답변 완성

1) 모니터링 결과 log-service의 디스크 사용률이 95%로 확인되어, 디스크 용량 부족 상태임을 확인했습니다. 이는 임계 수준(90% 이상)을 초과한 상태입니다.

2) 디스크 용량 확보를 위해 로그 파일, 임시 파일, 오래된 백업 등을 정리하여 여유 공간을 즉시 확보해야 합니다.

3) 만약 특정 프로세스가 비정상적으로 로그를 쌓고 있다면, 로그 레벨을 낮추거나 로테이션 주기를 단축하여 로그 생성량을 줄여야 합니다. 임시 조치로도 부족할 경우, 볼륨을 확장하거나 추가 스토리지를 연결하는 방안을 고려해야 합니다. 정상화 후에는 로그 보존 정책과 디스크 사용률 알림 임계값을 재검토하여 조기에 감지될 수 있도록 조정해야 합니다.

🔍 LLM에 전달된 최종 프롬프트 보기

04 · BEFORE / AFTER — 같은 입력

“order-service 배포하고 나서 에러가 엄청 나요”

Before 원본 키워드 미매칭 → **db-down** 폴백, DB failover 권고

- ⌛ 계획 수립: 할 일 1가지 (시스템: order-service)
- ⌛ 모니터링 지표 확인 (시스템: order-service)
- ⌛ 정보 부족 → 장애유형 "db-down"로 추가 조사
- ⌛ 대응 매뉴얼(runbook) 조회 (장애유형: db-down)
- ⌛ 답변 초안 작성 (시도 1)
- ⌛ 검수 통과
- ⌛ 최종 답변 완성

1) 모니터링 결과, order-service의 상태가 'degraded'로 확인되었으며, CPU 사용률은 62.5%, 메모리 사용률은 63.8%로 나타났습니다. 알림은 2건 발생했으며, 'HTTP 5xx error rate 12% (threshold 1%)'와 'error spike began right after deploy v2.3.1'이 보고되었습니다.

2) 현재 상황은 데이터베이스 연결 문제와 관련된 것으로 보이며, 이는 'DB Down 대응 절차'에 따라 처리해야 할 가능성이 있습니다. 주요 증상으로는 데이터베이스 연결 실패, 프라이머리 장애, 리플리케이션 중단, 연결 거부 등이 있습니다.

3) 대응 절차로는 모니터링 알람을 통해 장애를 인지하고, 커넥션 풀 및 리플리케이션 상태를 점검한 후 필요 시 failover 노드로 전환해야 합니다. 이후 서비스 정상화 여부를 확인하고, 원인 분석 및 재발 방지 대책을 문서화해야 합니다.

🔍 LLM에 전달된 최종 프롬프트 보기

After RAG가 **error-rate-spike** 판별 → 직전 버전 롤백 권고

- ⌛ 계획 수립: 할 일 1가지 (시스템: order-service)
- ⌛ 모니터링 지표 확인 (시스템: order-service)
- ⌛ 정보 부족 → 장애유형 "error-rate-spike"로 추가 조사 (rag 판별)
- ⌛ 대응 매뉴얼(runbook) 조회 (장애유형: error-rate-spike)
- ⌛ 답변 초안 작성 (시도 1)
- ⌛ 검수 통과
- ⌛ 최종 답변 완성

1) 모니터링 결과, order-service의 상태가 'degraded'로 확인되었으며, CPU 사용률은 57.8%, 메모리 사용률은 61.4%로 나타났습니다. 알림으로는 'HTTP 5xx error rate 12% (threshold 1%)'와 'error spike began right after deploy v2.3.1'이 수신되었습니다.

2) 에러율 급증 대응 절차에 따라, 에러율 상승이 배포 직후 시작된 것으로 확인되었으므로, 현재 배포된 v2.3.1 버전을 직전 버전으로 즉시 롤백하여 에러율이 감소하는지 확인해야 합니다.

3) 만약 롤백이 불가능한 경우, 문제 기능을 feature flag로 비활성화하거나 해당 엔드포인트의 트래픽을 차단하는 조치를 취해야 합니다. 이후 에러 로그의 스택트레이스를 분석하여 공통 예외 유형을 식별하고, 정상화 후에는 배포 파이프라인에 카나리 배포 또는 에러율 기반 자동 롤백 조건을 추가해야 합니다.

🔍 LLM에 전달된 최종 프롬프트 보기

04 · BEFORE / AFTER — 수치

같은 24건, 같은 모델로 실측

all_total_vue_student — zsh

```
$ python eval/compare.py openai-before openai-after-keyword openai-after
## 전체
| 지표 | openai-before | openai-after-keyword | openai-after |
|---|---|---|---|
| 라우팅 정확도 | 95.8% | 95.8% | 95.8% |
| 조사 대상 재현율 | 65.0% | 85.0% | 95.0% |
| 조사 대상 정밀도 | 65.8% | 74.9% | 100.0% |
| runbook 재현율 | 34.2% | 34.2% | 84.2% |
| runbook 정밀도 | 52.6% | 47.4% | 91.7% |
| 근거 없는 수치 포함률 | 0.0% | 0.0% | 0.0% |
| Judge 수치 인용(1~5) | 4.95 | 4.95 | 4.95 |
| Judge 조치 적합도(1~5) | 4.84 | 4.63 | 5.00 |
| 장애 질의 평균 지연(s) | 5.47 | 4.95 | 5.09 |
| 평균 작성 시도 횟수 | 1.00 | 1.00 | 1.00 |
| 총 비용(USD, 24건) | 0.0637 | 0.0676 | 0.0680 |
```

`eval/compare.py` 실행 결과 (가운데 열 = RAG만 끈 ablation)

검색 정확도 · runbook 재현율
34% → **84%**
정밀도 53% → 92%

조사 대상 재현율
65% → **95%**
정밀도 66% → 100%

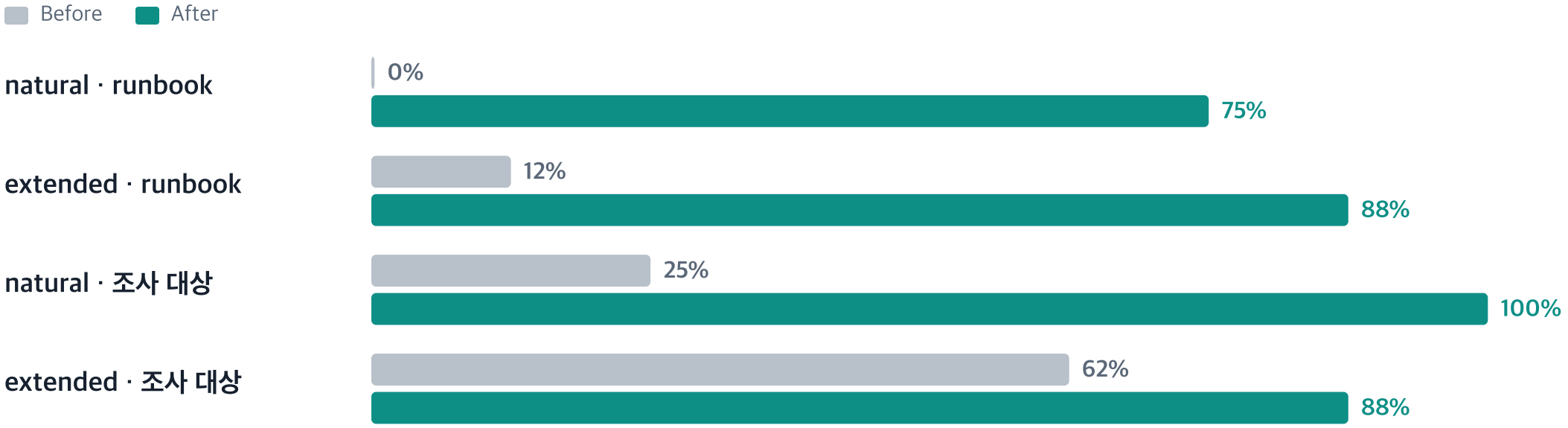
장애 질의 평균 지연
5.47s → **5.09s**
RAG 판별 1회 +233ms (편차 범위)

비용 (24건)
$0.0637 → **$0.0680**
+6.8% (runbook을 더 많이 읽음)

retry · 재작성
1 · 1.00 → **1 · 1.00**
MCP 재시도 횟수 · 평균 작성 시도

04 · BEFORE / AFTER — 세트별

어디서 좋아졌고, 그 대가는 얼마인가



233 ms

RAG 판별 1회 중앙값 (p90 273ms) · 키워드 0.34ms

$0.0000005

판별 1회 임베딩 비용 — 질의당 LLM 비용(약 $0.003) 대비 무시 가능

34%

RAG만 끄면 runbook 재현율이 원본과 같음 → 개선분은 전부 RAG 기여

05 · 한계

실측에서 드러난 실패 사례

사례	무슨 일이 있었나	원인 · 조치
게이트웨이 504 (B04 · N03)	"upstream timeout (504)"을 **error-rate-spike**로 판별 — 정답은 network-timeout	임베딩이 "504"를 5xx 에러와 가깝게 봄. runbook 문구를 평가셋에 맞추면 과적합이라 **고치지 않고 기록** → hybrid 검색 과제
billing-service (After v1)	미등록 시스템에 질문 문장만으로 runbook을 붙여 **없는 서비스에 롤백 권고**	평가로 발견 → 조사 대상이 전부 미등록이면 runbook 생략, 테스트 추가 (정밀도 86.8→91.7%)
"푸시 알림이 하나도 안 와요"	gpt-4o-mini 라우터가 잡담으로 분류 — Before/After 동일	라우팅은 이번 개선 범위 밖. 라우터 few-shot 보강 여지
근거 없는 수치	Before/After 모두 0건 — gpt-4o는 수치를 지어내지 않음	grounding 검증 효과는 **실측 불가**, 단위 테스트로만 확인. 소형 로컬 모델로 재측정 필요
LLM-judge	틀린 runbook을 쓴 답변에도 5점 — 점수가 천장에 붙음	정답 라벨 기반 지표(재현율·정밀도)를 주 지표로 사용

05 · 한계 및 트레이드오프

개선이 새로 만든 약점

영역	새 약점	대응 / 현재 상태
지연·비용	장애 질의마다 임베딩 호출 추가 (판별 1회 + 시스템 해석 최대 1회)	시스템 여러 개를 1회 batch로 묶고 인덱스 캐시. 비용은 무시 가능하나 왕복 약 230ms는 남음
임계값	top-1을 항상 채택 → 엉뚱한 질문도 가장 가까운 runbook에 억지 매칭	RAG_MIN_SCORE 제공(기본 0) — 라벨 데이터로 보정 필요
임베딩 의존	오프라인 hash 임베더는 영어 알림↔한국어 runbook 매칭 실패	임베딩 품질이 곧 판별 품질 — 다국어 로컬 임베딩으로 교체 여지
루프 안전성	runbook 여러 개 조회 · 반려 시 재작성 → 루프 경로 증가	replan 1라운드, runbook 최대 3개, 재작성 최대 3회 상한
평가셋	24건 · 본인 라벨링 · 1회 실행 — 통계적으로 약함	runbook에 알림 원문을 넣지 않아 누출 방지. 반복 실행·제3자 라벨 필요

다음 단계

Human-in-the-loop 승인(`interrupt()`) · research를 전문 서브에이전트로 분리(Multi-Agent) · LangSmith 추적 연결 · hybrid 검색(BM25 + 임베딩)

감사합니다

코드 · 평가 결과 · 문서: all_total_vue_student (README.md, eval/results, doc/)

광주_1반_G030_최재은